CS 486/686
Transformers & Attention

Yuntian Deng

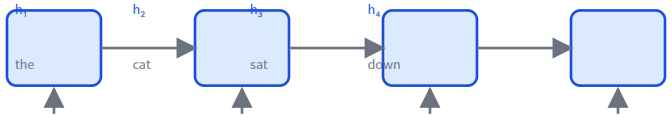
Lecture 21

RN 21.6 · "Attention Is All You Need" (Vaswani et al., 2017)

Learning goals

- Explain why **RNNs** struggle with long sequences.
- Derive **scaled dot-product attention** from queries, keys, values.
- Explain **multi-head attention** and **positional encodings**.
- Assemble a **transformer block** (attention + FFN + residual + norm).
- Distinguish **encoder** from **decoder** (causal masking).

Recall: RNNs process a sequence step by step



The state $h_t = f(h_{t-1}, x_t)$ carries information forward, one step at a time.

Three problems at scale:

- **Sequential:** step t waits for $t-1$ — can't parallelize over the sequence.
- **Long-range:** signal from far-back tokens fades (vanishing gradients).
- **Bottleneck:** everything must squeeze through one fixed-size state.

The idea: let every token look at every other

> **Attention** replaces the step-by-step chain with *direct* connections: to build a token's new representation, it reads a weighted mix of **all** tokens, choosing the weights based on relevance.

- Any token reaches any other in **one hop** — no fading.
- All positions are computed **in parallel** — GPU-friendly.

Step 1: tokens become vectors

Each of the n input tokens is mapped to a d-dimensional embedding. Stack them into a matrix:

$$X = \begin{bmatrix} — \ \mathbf{x}_1 \ — \\ \vdots \\ — \ \mathbf{x}_n \ — \end{bmatrix} \in \mathbb{R}^{n \times d}$$

Row i is token i's current representation. Attention will update every row.

Step 2: queries, keys, values

From X, three learned linear projections produce a **query**, **key**, and **value** for every token:

$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V$$

Query $\mathbf{q}_i$: "what am I looking for?"

Key $\mathbf{k}_j$: "what do I offer?"

Value $\mathbf{v}_j$: "what I pass on if attended to."

Token i attends to token j when query $\mathbf{q}_i$ matches key $\mathbf{k}_j$.

Step 3: scaled dot-product attention

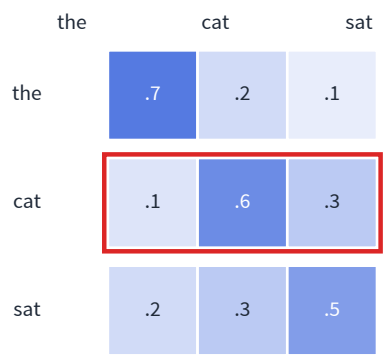
Score every query against every key (dot product), scale, softmax into weights, then mix the values:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

- $QK^\top \in \mathbb{R}^{n \times n}$: entry (i, j) = similarity of token i's query with token j's key.
- **softmax** over each row → nonnegative weights summing to 1.
- Why $\sqrt{d_k}$? Dot products grow with dimension; scaling keeps softmax out of its saturated, tiny-gradient regime.

What the weights look like

Three tokens. Each **row** of $\mathrm{softmax}(QK^\top/\sqrt{d_k})$ says how much a token attends to the others (rows sum to 1):



The red row: "**cat**" attends mostly to itself (0.6) and to "sat" (0.3).

Its new vector is the weighted sum of values:

$$\mathbf{z}_{\text{cat}} = 0.1\,\mathbf{v}_{\text{the}} + 0.6\,\mathbf{v}_{\text{cat}} + 0.3\,\mathbf{v}_{\text{sat}}$$

Multi-head attention

One attention pattern is limiting. Run h attention "heads" in parallel, each with its own W_Q, W_K, W_V, then concatenate and project:

$$\text{head}_i = \text{Attention}(XW_Q^i, XW_K^i, XW_V^i)$$
$$\text{MHA}(X) = \left[\text{head}_1; \ldots; \text{head}_h\right] W_O$$

- Each head learns a different relation (e.g. syntax, coreference, position).
- Each head works in a smaller subspace $d_k = d/h$, so the total cost is similar to one full-size head.

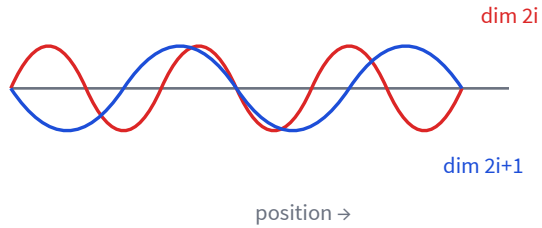
Where's the order? Positional encodings

Attention is a weighted sum — it's **permutation-invariant**: shuffle the tokens and the output just shuffles too. So we add position information to the embeddings.

Sinusoidal encoding (original transformer):

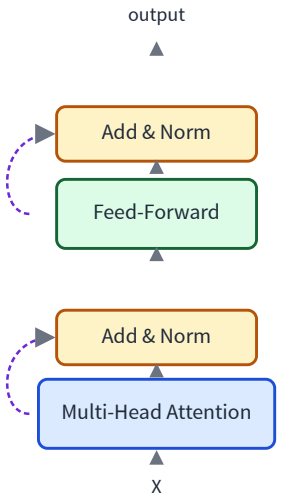
$$PE_{(pos,\,2i)} = \sin\left(pos / 10000^{2i/d}\right)$$

$$PE_{(pos,\,2i+1)} = \cos\left(pos / 10000^{2i/d}\right)$$



Add PE to each token's embedding. Modern models often use **learned** positions or **RoPE** (rotary).

The transformer block



Each block = two sublayers, each wrapped in a **residual** connection and **layer norm**:

- **Attention**: tokens exchange information.
- **Feed-forward**: a per-token MLP adds nonlinear processing.
- **Residuals** (dashed) let gradients flow; **norm** keeps activations stable.
- **Stack** N of these blocks → a deep transformer.

Encoder, decoder, and causal masking

Encoder

Every token attends to **all** tokens (bidirectional). Good for understanding a whole input.

Decoder

A token may attend only to **earlier** tokens — so it can generate the next one without peeking ahead.

The **causal mask** sets future scores to $-\infty$ before the softmax, zeroing those weights.

The cost of looking everywhere

> Every token attends to every token → the score matrix $QK^\top$ is $n \times n$. Cost is
>
> $$O(n^2 d)$$
>
> — quadratic in sequence length n.

- Great trade for RNNs' sequential bottleneck: attention is fully **parallel**.
- But long contexts are expensive — a very active research area (efficient / sparse attention).

Why transformers took over

Parallel

All positions at once — trains fast on GPUs/TPUs.

Long-range

Any token reaches any other in one hop.

Scales

Add data + parameters and it keeps improving.

The same block powers vision, audio, protein folding — and every large language model.

Learning goals (recap) — Next: LLMs

✓ Why **RNNs** struggle with long sequences.

✓ **Scaled dot-product attention** from queries, keys, values.

✓ **Multi-head attention** and **positional encodings**.

✓ The **transformer block**; encoder vs decoder (causal masking).

L22: stack decoder blocks + train at scale → **Large Language Models**.